

Programação

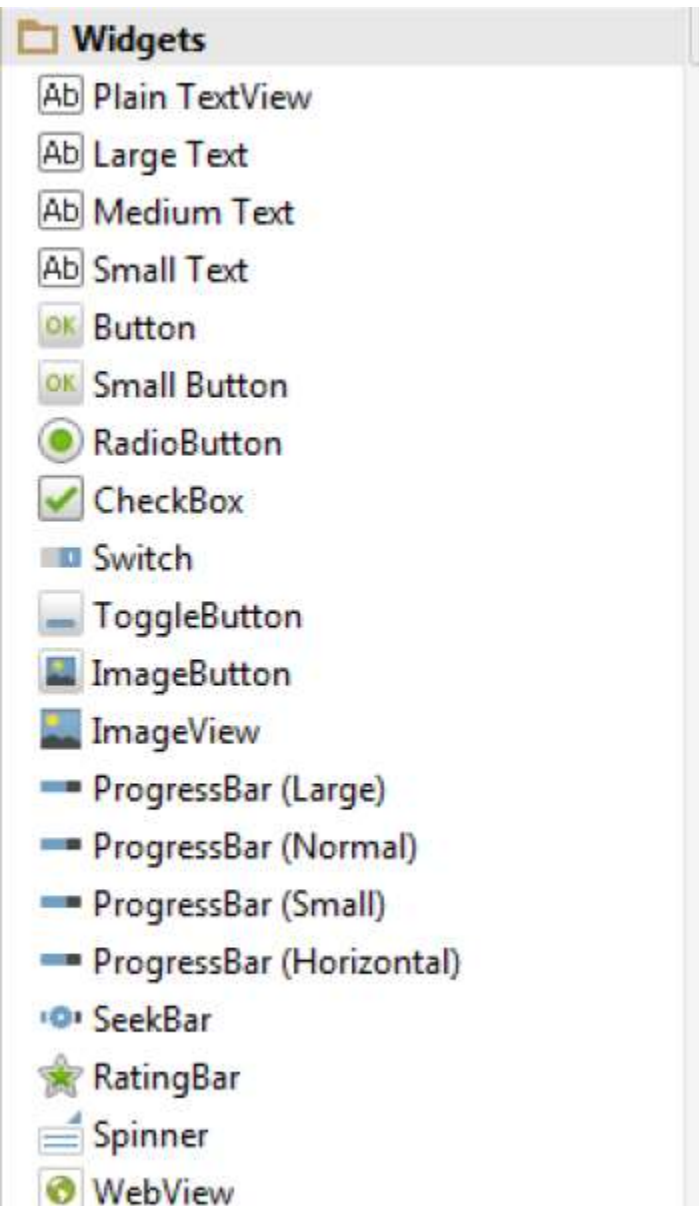
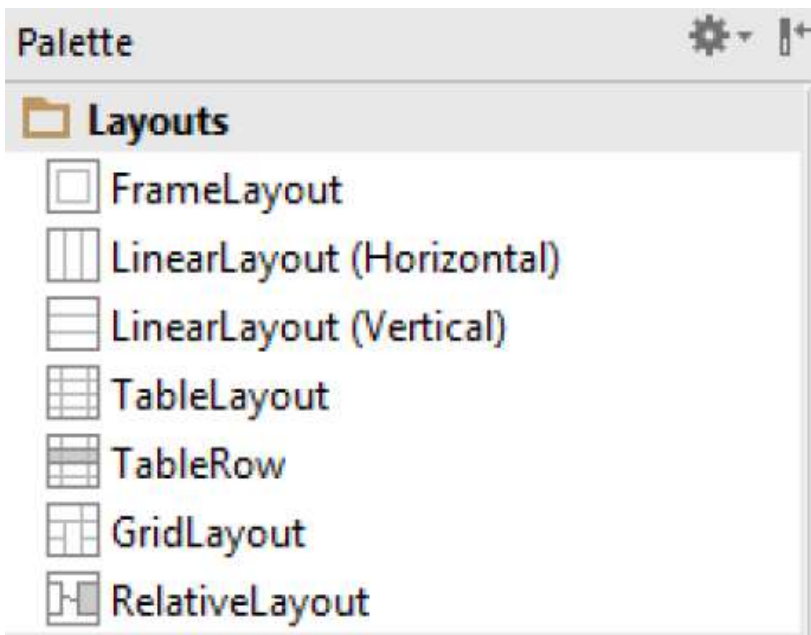
Dispositivos Móveis

aula 05



- A ferramenta de desenvolvimento do Android SDK oferece um conjunto de componentes gráficos (layouts, containers, widgets, etc.) que podem ser usados em uma aplicação a ser construída.
- Os componentes estão distribuídos em diversas seções na paleta de componentes.

Paleta de Componentes



- **TextView** - usado para mostrar textos na tela.
- **EditText** - usado para que o usuário digite alguma entrada.
- **Button** - usado para que o usuário execute ações através de cliques.
- **CheckBox** - usado para que o usuário marque opções.
- **RadioButton** – o mesmo que acima, mas permite marcar apenas uma opção.
- **Spinner** - usado para que o usuário selecione uma opção (combo box).
- **ImageView** - usado para mostrar imagens na tela.

- Um TextView é usado para exibir algum texto para o usuário.
- O valor do atributo "text" é o texto que será exibido para o usuário.
- Ele está disponível em quatro estilos (Plain , Large, Medium e Small).

```
<TextView  
    android:layout_width = "wrap_content"  
    android:layout_height = "wrap_content"  
    android:id = "@+id/texto"  
    android:text = "Texto" />
```

- Para maiores informações:

<http://developer.android.com/reference/android/widget/TextView.html>

- A quantidade máxima de linhas de um TextView pode ser definida através do atributo "maxLines". O valor desse atributo deve ser um número inteiro maior do que zero.
- Se o texto for maior do que o espaço de um TextView, podemos utilizar o atributo "ellipsize" para abreviá-lo com o uso de reticências (...).
- Os principais valores que podem ser associados a esse atributo são: "start", "middle" e "end". A diferença entre esses valores é o posicionamento das reticências.

```
<TextView
    android:layout_width = "60 dp"
    android:layout_height = "wrap_content"
    android:ellipsize = "end"
    android:maxLines = "1"
    android:text = "Texto muito longo" />
```

- Os elementos XML precisam obrigatoriamente dos atributos “layout_width” e “layout_height” para indicar a largura e altura de cada elemento.
- Possíveis valores para esses atributos:
 - ✓ **match_parent** - Esse valor faz o elemento ter a mesma largura ou altura do elemento pai.
 - ✓ **wrap_content** - Esse valor faz o elemento ter a largura ou a altura suficiente englobar o seu conteúdo.
 - ✓ **fill_parent** - Mesmo que match_parent. Recomenda-se utilizar o valor match_parent a partir da API Level 8.
 - ✓ **valor** - Uma dimensão, ou seja, um valor numérico utilizando uma das unidades de medida suportadas na plataforma Android.
Por ex: valores absolutos (em pixels)

- O atributo "id" é usado para identificar os elementos de forma única.
- Apesar de não ser obrigatório definir este atributo de forma explícita, é muito comum adicioná-lo nos elementos.
- Esse atributo é muito importante para recuperar os elementos no código Java ou para organizar os elementos dentro de um "RelativeLayout" que é um tipo específico de ViewGroup (container).
- Por serem considerados recursos, os "ids" são registrados na classe especial R.

- O valor do atributo "id" deve:
 1. Começar com o caractere "arroba" (@).
 2. O sinal de "mais" (+) pode aparecer depois do "arroba" (@).
 - Esse sinal indica que estamos criando um novo "id".
 - Contudo, podemos utilizá-lo mesmo quando queremos referenciar um "id" existente.
 3. Em seguida, aparecem os caracteres "id/".
 4. Por fim, aparece o "id" propriamente.

- Por exemplo, o “id” do “TextView” do exemplo abaixo poderia ser acessado com o atributo “R.id.texto” no código Java.

```
<TextView  
    android:layout_width = "wrap_content"  
    android:layout_height = "wrap_content"  
    android:id = "@+id/texto"  
    android:text = "Texto" />
```

- Um "Button" é utilizado para exibir um botão para o usuário.
- É possível adicionar uma imagem nos botões com os atributos "drawableLeft", "drawableRight", "drawableTop" ou "drawableBottom".
- Também é possível definir uma imagem ou uma cor como background de um botão utilizando o atributo "background".
- Esse atributo também pode ser utilizado nos demais elementos gráficos.

```
<Button
    android:id = "@+id/ok"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:background = "#ffff00"
    android:drawableLeft = "@drawable/confirma"
    android:text = "Ok" />
```

- É possível definir como o usuário pode interagir com a tela, clicando e digitando nos objetos gráficos (“*widgets*”).
- No código Java de uma “activity”, é possível definir a lógica que deve ser executada quando um determinado botão for pressionado.
- Isso é feito através de “*listeners*” de eventos.

- Para utilizar um *widget* definido no layout, basta recuperá-lo ("buscá-lo") e fazer referência no código java.
- Para isso, é utilizado o método **findViewById()**.
- O "id" do botão deve ser passado como parâmetro para esse método.
- Lembre-se que, no código Java, os "ids" dos elementos são recuperados através da classe especial R.

- É necessário fazer um “*cast*” para o tipo de elemento que queremos, porque o **findViewById** sempre retorna um objeto do tipo **View**.
- O parâmetro que o método recebe, se refere ao “id” que foi dado ao elemento no XML.
- Lembre-se que o arquivo **R.java** faz a ligação entre XML e código java.
- O arquivo **R.java** é quem guarda a referência de todos os layouts e elementos presentes no XML.

```
Button button = (Button)findViewById(R.id.see_message_button);
```

- Após termos o objeto, podemos adicionar *listeners*, como no exemplo abaixo:

```
Button button = (Button) findViewById(R.id.see_message_button);

button.setOnClickListener (new OnClickListener() {
    @Override
    public void onClick (View arg0) {

        // faz alguma coisa

    }
});
```

- Dessa maneira, sempre que o botão for clicado, o método **onClick** será chamado.
- Existem vários outros tipos de "*listeners*", todos sempre começam com o sufixo "*on*" indicando depois qual tipo de evento eles estão "escutando".
- Exemplos:

Button: `setOnClickListener` – evento: `OnClick`

EditText: `setOnEditorActionListener` – evento: `OnEditorAction`

- O Toast fornece um feedback simples e rápido ao usuário em uma pequena janela.
- Essa janela é exibida na quantidade de espaço necessário para a mensagem que lhe foi determinada, permanecendo visível por um tempo limite.

```
Context contexto = getApplicationContext();
String texto = "exemplo toast";
int duracao = Toast.LENGTH_SHORT;
Toast toast = Toast.makeText (contexto, texto, duracao);
toast.show();
```

- Tal como o nome sugere, permite que objetos recém-criados entendam o que vem acontecendo.
- Normalmente você utiliza "context" para obter informações sobre uma outra parte de seu programa (activity, package/application).
- Você pode obter o "context" invocando os métodos:
 - `getApplicationContext ()`,
 - `getContext ()`,
 - `getBaseContext ()`
 - `this` (quando você está classe da activity).

```
Button botaoOk = (Button) findViewById (R.id.ok);

botaoOk.setOnClickListener (new View.OnClickListener() {
    @Override
    public void onClick (View v) {
        Toast toast = Toast.makeText(getApplicationContext(),
            "Processando ... ", Toast.LENGTH_SHORT);
        toast.show();
    }
});
```

```
<Button
    android:id = "@+id/ok"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:background = "#ffff00 "
    android:drawableLeft = "@drawable/confirma"
    android:text = "Ok" />
```

- Existe outra forma para definir a lógica que deve ser executada quando um determinado botão é pressionado.
- Podemos criar um método público na classe da "activity" com um parâmetro do tipo "View" e com retorno "void".
- Depois, associar esse método ao botão através do atributo "onClick".
- Toda vez que o botão for pressionado esse método será executado.

```
public void onClick (View view) {
    Toast toast = Toast.makeText (getApplicationContext(),
                                   "Processando...", Toast.LENGTH_SHORT);
    toast.show();
}
```

```
<Button
    android:id = "@+id/ok"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:text = "Ok"
    android:onClick = "onClick"/>
```

- Para maiores informações:

<http://developer.android.com/reference/android/widget/Button.html>

- Um EditText é um tipo específico de TextView que permite que o usuário modifique o seu texto.
- O atributo "hint" é utilizado para definir uma dica para o usuário saber o que deve ser preenchido em um EditText.
- O atributo "inputType" determina o tipo de texto de um EditText.
- O teclado oferecido ao usuário para modificar o texto de um EditText depende do valor desse atributo.

- Alguns valores que podem ser usados no atributo "inputType":
 - **text** - texto normal
 - **textEmailAddress** - email
 - **textPassword** - senha
 - **textAutoCorrect** - texto normal com correção ortográfica
 - **textCapSentences** - texto normal com a primeira letra de cada sentença maiúscula
- Dois ou mais valores podem ser utilizados ao mesmo tempo no atributo inputType. Para isso, basta concatená-los com o caractere barra vertical (|).

```
<EditText
    android:id = "@+id/mensagem"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:hint = "Escreva uma mensagem"
    android:inputType = "textAutoCorrect|textCapSentences" />
```

- Para recuperar o texto de um EditText, podemos utilizar o método `getEditableText()`.
- Esse método devolve a referência de um objeto "Editable".
- O texto pode ser extraído através do método `toString()`.

```
EditText nome = (EditText) findViewById (R.id.mensagem);
String texto = nome.getEditableText().toString();
```

- Uma ação pode ser definida no teclado correspondente a um EditText através do atributo "imeOptions;".
- Os valores "actionNext", "actionDone", "actionSend" e "actionSearch" são exemplos de valores aceitos por esse atributo.

```
<EditText
    android:id = "@+id/ mensagem"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:hint = "Escreva uma mensagem"
    android:imeOptions = "actionSend"
    android:inputType = "textAutoCorrect|textCapSentences" />
```

- No código Java da “activity”, podemos definir a lógica que deve ser executada quando a ação definida no teclado de um EditText é disparada.
- Ex: um objeto da interface `TextView.OnEditorActionListener` foi associado a um EditText.
- O método `onEditorAction()` é chamado toda vez que a ação do EditText é disparada.

```
EditText editText = (EditText) findViewById (R.id.mensagem);

editText.setOnEditorActionListener (new TextView.OnEditorActionListener() {

    @Override
    public boolean onEditorAction (TextView v, int actionId, KeyEvent event){
        boolean handled = false ;
        if (actionId == EditorInfo.IME_ACTION_SEND) {
            Toast toast = Toast.makeText (getApplicationContext(),
                                           "Enviando ... ", Toast.LENGTH_SHORT);

            toast.show ();
        }
        return handled ;
    }
});
```

- Os “RadioButtons” permitem que o usuário selecione uma opção de um conjunto de opções.
- O “RadioGroup” é utilizado para definir conjuntos de opções mutualmente exclusivas.
- O atributo “orientation” de um “RadioGroup” determina se as opções do conjunto correspondente aparecerão na horizontal ou na vertical.
- O atributo “checked” pode ser utilizado para indicar se um “RadioButton” está selecionado ou não.
- Dois ou mais “RadioButtons” em um mesmo “RadioGroup” não podem estar selecionados ao mesmo tempo.

```
<RadioGroup
    android:id = "@+id/ sexo"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:orientation = "horizontal" >
    <RadioButton
        android:checked = "true"
        android:id = "@+id/feminino"
        android:layout_width = "wrap_content"
        android:layout_height = "wrap_content"
        android:text = "Feminino" />
    <RadioButton
        android:id = "@+id/ masculino"
        android:layout_width = "wrap_content"
        android:layout_height = "wrap_content"
        android:text = "Masculino" />
</RadioGroup>
```

- No código Java da "activity", para recuperar a opção selecionada pelo usuário, podemos utilizar o método `findViewById()` para obter o "RadioGroup" que define o conjunto de opções.
- Em seguida, usar o método `getCheckedRadioButtonId()` para descobrir o id do "RadioButton" selecionado nesse "RadioGroup".
- Esse método devolve -1 se nenhum "RadioButton" estiver selecionado.
- Por fim, novamente, podemos utilizar o método `findViewById()` para recuperar esse "RadioButton".

```
RadioGroup radioGroup = (RadioGroup) findViewById (R.id.sexo );  
  
int radioButtonId = radioGroup.getCheckedRadioButtonId();  
  
RadioButton radioButton = (RadioButton) findViewById (radioButtonId);  
  
Toast toast = Toast.makeText (getApplicationContext(),  
                             radioButton.getText(), Toast.LENGTH_SHORT);  
toast.show();
```

- Para maiores informações:

<http://developer.android.com/reference/android/widget/RadioButton.html>

- Os CheckBoxs permitem que o usuário selecione uma ou mais opções de um conjunto de opções.
- O atributo "checked" pode ser utilizado para indicar se um CheckBox está selecionado ou não.

- Para maiores informações:

<http://developer.android.com/reference/android/widget/Checkbox.html>

<CheckBox

```
    android:checked = "true"
    android:id = "@+id/java"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:text = "Java" />
```

<CheckBox

```
    android:id = "@+id/ csharp"
    android:layout_width = "match_parent "
    android:layout_height = "wrap_content"
    android:text = "C#" />
```

<CheckBox

```
    android:checked = "true"
    android:id = "@+id/php"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:text = "PHP" />
```

- No código Java, para saber o estado de um determinado "Checkbox", podemos utilizar o método "findViewById()" para recuperá-lo e o método "isChecked()" para verificar o seu estado.

```
CheckBox checkBox = (CheckBox)findViewById(R.id.java);

boolean checked = checkBox.isChecked();

Toast toast = Toast.makeText (getApplicationContext(),
                             checkBox.getText() + ":" + checked,
                             Toast.LENGTH_SHORT);

toast.show ();
```

- Um Switch permite que o usuário selecione uma entre duas opções (off e on).
- Os atributos textOff e textOn permitem alterar os textos das opções off e on respectivamente.

```
<Switch
    android:id = "@+id/musica"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:text = " Música"
    android:textOff = "Desligada"
    android:textOn = "Ligada"/>
```

- No código Java, para saber o estado de um determinado Switch, podemos utilizar o método `findViewById()` para recuperá-lo e o método `isChecked()` para verificar o seu estado:

```
Switch s = (Switch)findViewById(R.id.musica);
```

```
boolean checked = s.isChecked();
```

```
Toast toast = Toast.makeText(getApplicationContext(),  
                                s.getText() + ":" + checked,  
                                Toast.LENGTH_SHORT);  
toast.show();
```

- Um ImageButton é análogo a um Button. Contudo, um ImageButton possui apenas imagem.

```
< ImageButton  
    android:layout_width = "match_parent"  
    android:layout_height = "wrap_content"  
    android:src = "@drawable/confirma"/>
```

- Um ImageView é utilizado para exibir para o usuário uma imagem.
- O atributo src determina o recurso correspondente à imagem que deve ser exibida por um ImageView.
- O atributo scaleType determina como a imagem deve se adaptar à área do ImageView.
- Para esse atributo, podemos utilizar os seguintes valores:

center - A imagem fica centralizada na área do ImageView.

- O tamanho original da imagem não sofre alteração.
- Se a imagem for maior que a área do ImageView parte dela não aparecerá.

centerCrop - A imagem é centralizada na área do ImageView.

- O seu tamanho aumenta ou diminui sem perder a proporção original de tal forma que a imagem ocupe toda a área do ImageView.
- Além disso, pelo menos uma das dimensões (altura ou largura) da imagem será exatamente igual à dimensão correspondente na área do ImageView.
- Normalmente, nesse processo, a imagem é “cortada”.

centerInside - centralizada na área do ImageView.

- Se a imagem é menor do que a área do ImageView, o tamanho da imagem não sofre alteração.
- Caso contrário, a imagem diminui sem perder a proporção original até que as suas dimensões sejam menores ou iguais às dimensões correspondentes na área do ImageView.
- Nesse processo, a imagem sempre aparecerá sem “cortes”.

fitCenter - A imagem fica centralizada na área do ImageView.

- O seu tamanho aumenta ou diminui sem perder a proporção original e sem exceder as dimensões do ImageView.
- Além disso, a imagem terá o maior tamanho possível. Nesse processo, a imagem sempre aparecerá sem “cortes”.

- **fitEnd** - Semelhante ao fitCenter. Contudo, a imagem é acomodada no canto inferior direito da área do ImageView.
- **fitStart** - Semelhante ao fitCenter. Contudo, a imagem é acomodada no canto superior esquerdo da área do ImageView.
- **fitXY** - As dimensões da imagem serão iguais às dimensões do ImageView. Nesse processo, a proporção original da imagem pode não ser preservada.
- **matrix** - Permite que o tamanho da imagem aumente ou diminua com proporções personalizadas.

```
<ImageView  
    android:id = "@+id/logotipo"  
    android:layout_width = "200dp"  
    android:layout_height = "80dp"  
    android:scaleType = "center"  
    android:src = "@drawable/imagem_logo" />
```

Para maiores detalhes sobre o ImageView consulte a documentação:
<http://developer.android.com/reference/android/widget/ImageView.html>

- Um Spinner permite que o usuário selecione uma opção de um conjunto de opções.
- As opções podem ser inseridas em um Spinner de forma estática ou dinâmica.
- Na forma estática, podemos definir um “array de strings” no arquivo strings.xml utilizando o elemento “string-array”.
- No exemplo a seguir, um nome foi definido para o array e as opções foram definidas com o elemento “item”.
- As opções definidas em um “array de strings” no arquivo strings.xml podem ser associadas a um Spinner através do atributo “entries”.
- O array é acessado com o prefixo “@array/” seguido do seu nome.

```
<Spinner
    android:id = "@+id/cidades"
    android:layout_width = "match_parent"
    android:layout_height = "wrap_content"
    android:entries = "@array/cidades" />
```

```
<resources >
    . . .
    <string - array name = "cidades">
        <item>São Paulo</item >
        <item>Mogi das Cruzes</item >
        <item>Campinas</item >
        <item>Pindamonhangaba</item >
    </string - array>
</resources>
```

- Outra possibilidade é definir as opções de um Spinner dinamicamente. Por exemplo, com dados obtidos de um web service.
- Para isso, é necessário criar um “Adapter” para fornecer as opções para o Spinner.
- No código Java a seguir, um Spinner foi recuperado através do método “findViewById()”.
- Uma lista de strings foi criada e populada com nomes de algumas cidades.
- Um “ArrayAdapter” de string foi criado a partir dessa lista.

- O segundo parâmetro do construtor usado da classe “ArrayAdapter” é o layout que será aplicado a cada opção do Spinner.
- No exemplo, foi usado um layout fornecido pela plataforma Android, o “R.layout.simple_spinner_dropdown_item”.
- Por fim, o “Adapter” foi associado ao Spinner.

```
Spinner spinner = (Spinner) findViewById (R.id.cidades);
ArrayList <String> cidades = new ArrayList < >();
cidades.add ("São Paulo");
cidades.add ("Mogi das Cruzes");
cidades.add ("Campinas");
cidades.add ("Pindamonhangaba");
ArrayAdapter <String> adapter = new ArrayAdapter <String>(this,
    android.R.layout.simple_spinner_dropdown_item,cidades);
spinner.setAdapter (adapter);
```

- A opção selecionada em um Spinner pode ser recuperada através do método `getSelectedItem()`.

```
Spinner cidades = (Spinner) findViewById(R.id.cidades);  
string cidade = cidades.getSelectedItem().toString();
```

Para maiores detalhes sobre o Spinner consulte a documentação:
<http://developer.android.com/reference/android/widget/Spinner.html>